

Predictive alerting for cloud metrics - Report

Viktor Ćurčić

1 Task Definition

The goal of this project was to design and implement a model that predicts whether an incident will occur within the next H time steps based on the previous W steps of one or more time-series metrics.

The task was framed as a predictive alerting problem for cloud-service monitoring. In that setting, the system continuously observes operational metrics such as CPU utilization, memory usage, request rate, and error rate, and attempts to raise an alert before an incident begins.

The key challenge is that alerting is not the same as ordinary classification. In a real monitoring system, a good model must do more than achieve high sample-level accuracy. It must: detect incidents before they start; provide useful lead time; avoid excessive false alerts; and remain interpretable enough to support operational decisions.

The ideas used in the project solution are in some part based on my personal experiences in cybersecurity alert and attack detections as an L1 cybersecurity analyst.

2 Problem Formulation

The problem was formulated as a supervised learning problem of binary classification. The idea behind it reminded me of Wald's test in hypothesis testing as both require the samples to arrive in time and not to be available at once. The idea is as follows: in each time step t the input consists of the previous W time steps of metrics and the label indicates whether an incident starts within the next H time steps.

For this project, the values $W = 60$ minutes and $H=10$ minutes were used, so each prediction used the previous 60 minutes of metric history to estimate the probability that an incident would begin within the next 10 minutes. In cybersecurity, quite longer intervals are used (sometimes weeks), though in this instance that would require millions of data samples, so for the purposes of simplicity, the values were lowered.

An also important design choice is whether to use incident starts or incident overlaps, or rather should the positive class be defined by its start of its duration. If labels are defined by any overlap with a future incident interval, then many windows close to an already ongoing incident become positive. That inflates the number of positive samples and makes the task less aligned with real alerting. As such, the use of incident starts is more informative and precise.

3 Dataset

For this task, a synthetic dataset of more than 120000 samples was used, as the task allows it and the goal is problem formulation and evaluation, not dataset complexity. The dataset incorporates four cloud-like metrics: CPU utilization, memory usage, request rate, error rate. In my dataset, the percentage of positive windows only stood at $\approx 2\%$, reflecting real-world situations. In cybersecurity, 98% of all alerts on average represent false alarms.

This type of dataset poses another problem for metric evaluation - we have a highly unbalanced dataset. Models could, therefore, achieve quite high accuracies by just predicting "no incident. Because of that, average precision and recall were used as they were more informative.

As I have pointed out, four metrics were used for to describe the signals. In real world systems, incident can be preceded by suspicious signals: rising CPU usage, increased latency and spikes in error rates. But these precursors don't have to lead to incidents. Because of that, four types of incidents were modeled: detectable incidents (can be easily distinguished), weak precursor incidents (the precursors are not exaggerated enough for the incident to be classified as detectable), unpredictable incidents (could not be inferred from the precursors) and false precursor patterns (true incident pattern, but actually a

false alarm). These incidents make up the dataset: 70% detectable incidents with clearer precursors, 20% hard incidents with weaker precursors and 10% unpredictable incidents with no clear precursor, with different false precursor patterns injected as well. That was important so the task would not be unrealistically easy.

As I have said, this was done for the task to not be unrealistically easy. In my first iteration of the task, only the detectable incidents were used and models achieved 95% average precision, which is realistically too good.

4 Modeling Choices

The first model choice was rule-based alerts. I implemented simple static rules such as CPU > 85, error rate > 3, CPU rising quickly over the last few minutes and memory high and still rising. This baseline represents classical threshold alerting. The second was logistic regression. I used it as a simple supervised machine learning baseline on engineered window features. Finally, I used XGBoost. I used it as my final model as it handles nonlinear relationships well, works well on tabular engineered features, is robust and fast and is easier to explain and deploy than sequence deep-learning models. More complex sequence models such as LSTMs or Transformers were considered conceptually, but not used in the final prototype, because the dataset was moderate in complexity, formulation and evaluation were emphasized as opposed to architecture size and tree-based models already worked well.

I used engineered features extracted from each 60-minute window, such as mean, standard deviation, minimum, maximum, change over window etc. which tallied to 43 features per window. This part changed several times through the project: first version had raw flattened windows only, the second version had a large engineered feature set, and the final version had a smaller, more focused engineered feature set. The final version was better because it kept the model expressive enough while making the pipeline simpler and easier to explain.

As for the training and split setup, I used a time-based split, not a random split, to avoid leakage - training 60%, validation 20% and test 20%. This is essential for time-series alerting, because the model should be trained on past data and evaluated on future data.

The model outputs probabilities, but an alerting system needs a binary decision - alert or no alert. The threshold for converting probability into an alert is a key decision. A low threshold gives higher recall, earlier detection but more false alarms whereas a high threshold gives fewer false alarms, but lower recall and more missed incidents. This is similar in Wald's test with its threshold choices. Initially, thresholds were chosen using only window-level recall targets. That turned out to be insufficient, because good sample-level recall did not necessarily translate into useful incident-level performance. This was one of the most important insights of the project.

The final version selected the threshold using incident-level validation metrics, not just window-level classification metrics. Window-level metrics were mainly used for model comparison - average precision, recall, F1. Incident-level metrics were more important for a real alerting system - fraction of incidents detected before start, mean lead time, median lead time, false alarms per day.

5 Results

5.1 Rule-based Alerts

This baseline performed poorly - average precision 0.0299, recall at best F1 threshold: 4.98%, precision at best F1 threshold 22.02%. This result was useful because it demonstrated that simple threshold logic alone is insufficient for this predictive alerting task.

5.2 Logistic Regression

The logistic regression model served as a simple machine-learning baseline on engineered window features. Its validation Average Precision was 0.4345. This was a substantial improvement over the rule-based baseline, showing that supervised learning over sliding-window features can capture useful predictive structure.

5.3 XGBoost

The final XGBoost model achieved:

- Validation Average Precision: 0.5592
- Test Average Precision: 0.7412
- Test Recall: 79.32%
- Test Precision: 36.51%
- Test F1 Score: 0.500

Thus, XGBoost clearly outperformed both baselines and provided the strongest predictive ranking of risky windows.

5.4 Threshold Selection

Because the model outputs probabilities rather than direct binary alerts, a threshold had to be chosen to convert predicted risk into alert decisions.

As I have pointed out, major design improvement during the project was moving from threshold selection based only on window-level recall to threshold selection based on incident-level validation metrics. The target was to obtain approximately 70% incident detection while keeping false alarms at a manageable level.

At threshold 0.5, the validation-set incident-level results were:

- Incidents in validation period: 41
- Detection rate: 51.2%
- False alarms per day: 3.6
- Mean lead time: 6.2 minutes

After threshold search, the selected threshold was 0.3439. To better understand the operational trade-off, a threshold sensitivity analysis was performed:

Threshold	Incident Detection	False Alarms/Day	Mean Lead Time
0.25	48.8%	9.3	5.8 min
0.35	56.1%	5.3	5.9 min
0.45	56.1%	4.1	6.2 min
0.55	48.8%	3.3	6.2 min
0.65	31.7%	2.6	5.7 min

This table illustrates the expected operational trade-off - lower thresholds increase detection but also increase alert volume while higher thresholds reduce false alarms at the cost of missed incidents.

5.5 Window-Level Evaluation

At the selected threshold of 0.3439, the final XGBoost model achieved the following window-level test performance: recall 79.32%, precision 36.51%, F1 Score 0.500, average precision 0.7412. These results indicate that the model can rank high-risk windows substantially better than the baselines.

5.6 Incident-Level Evaluation

Since the practical goal of the system is to detect incidents before they begin, incident-level evaluation was also performed on the held-out test period. The measured quantities were incident detection rate, lead time, and false alarms per day after applying a cooldown policy.

The final incident-level test results were:

- Incidents starting in test period: 44
- Detected before start: 26
- Incident detection rate: 59.1%

- Mean lead time: 6.0 minutes
- Median lead time: 6.5 minutes
- False alarms: 118
- False alarms per day: 6.6

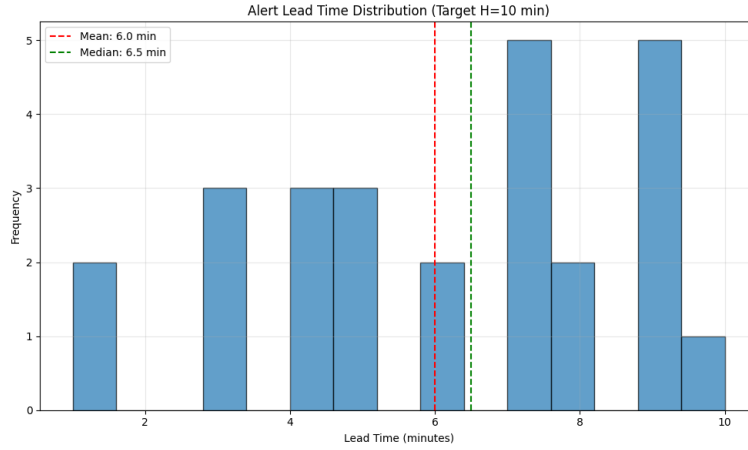


Figure 1

These metrics provide a more realistic view of alerting usefulness than window-level metrics alone. In particular, the model often produced alerts shortly before incident onset, but still missed a substantial fraction of incidents and generated a nontrivial number of false alarms.

6 Key Insights

One of the most important lessons of the project was the difference between window-level and incident-level evaluation. As I have mentioned, window-level evaluation treats each sliding window as an independent example. This is useful for comparing models statistically. However, alerting systems care about events rather than isolated windows.

This explains why window-level recall and incident-level detection rate are not the same. A model may classify many positive windows correctly but still miss the earliest useful warning window for an incident. Conversely, detecting only one correctly timed window may already be enough to count the incident as successfully detected.

In the final results, this difference was visible in the gap between window-level recall: 79.3% and incident-level detection rate: 59.1%. This gap is one of the key findings of the project.

In terms of model selection, many insights were found as well. The rule-based baseline was too rigid - it could only capture simple threshold conditions and missed many incidents. Logistic regression was much stronger, which showed that the engineered features already contained significant predictive information. However, logistic regression assumes approximately linear relationships between features. XGBoost performed best because it can model nonlinear interactions and threshold-like effects more naturally. For example, incident risk may become high only when several conditions hold simultaneously, such as rising CPU, increasing memory, and elevated error-related behavior. Such interactions are easier for tree-based models to learn than for linear models.

The mean lead time of about 6 minutes indicates that the system often raises alerts shortly before incident onset. This is operationally useful, but still limited. In some applications, 6 minutes may be enough for mitigation; in others, a longer warning would be preferable. The limited lead time suggests that the model mostly detects relatively late precursors rather than very early warning signs. Improving lead time would likely require better representation of long-term degradation patterns, richer features or different modeling approaches.

The threshold sensitivity analysis made clear that alerting is not only a prediction problem but also an operations problem. In safety-critical systems, higher recall may justify higher alert volume. In cost-sensitive operations, lower alert volume may be preferred even at the expense of missed incidents. This was one of the strongest practical insights from the project.

7 Limitations

Although the final system is a useful prototype, it still has several limitations. The dataset is synthetic and does not capture all real-world cloud-service behaviors. The system models a simplified single-stream setting, whereas real deployments may involve many services and dependencies. The threshold is fixed after validation and may require recalibration over time. The average lead time is relatively short. Some incidents are intentionally unpredictable, which places an upper bound on achievable recall. Finally, the implementation does not yet include explicit online retraining or concept-drift adaptation.

8 Adaptation to a Real Alerting System

The prototype can be adapted to a real alerting environment in a straightforward way. First, synthetic metrics would be replaced by real monitoring data from systems such as CloudWatch, Prometheus, or Datadog. Incident labels could be obtained from PagerDuty or different incident-management tickets.

Second, the training pipeline would periodically rebuild sliding windows from recent data, retrain the model, validate thresholds, and store the resulting artifacts for inference.

Third, a lightweight inference service would continuously fetch the latest W minutes of metrics, compute the same engineered features, score the model, and raise alerts if the probability exceeds the chosen threshold.

Finally, the deployed system should include cooldown, deduplication and monitoring of the model itself. Important operational metrics would include drift in prediction scores, incident detection rate, alert volume, false alarms per day, and lead-time distribution.

9 Conclusion

This project implemented a predictive alerting prototype for cloud-service incidents using a sliding-window supervised-learning formulation.

The final system used the previous 60 minutes of metric history, predicted whether an incident would start in the next 10 minutes, compared rule-based alerts, logistic regression, and XGBoost, used chronological train/validation/test splits, tuned alert thresholds using incident-level validation and evaluated both window-level and incident-level performance.

The final XGBoost model substantially outperformed the rule-based and logistic-regression baselines. The main conclusion is that predictive alerting should not be judged purely as a binary classification task. The true objective is event-level usefulness - detecting incidents early enough and with sufficiently low alert noise to be operationally valuable.